



ENG100 PROJECT

MATLAB

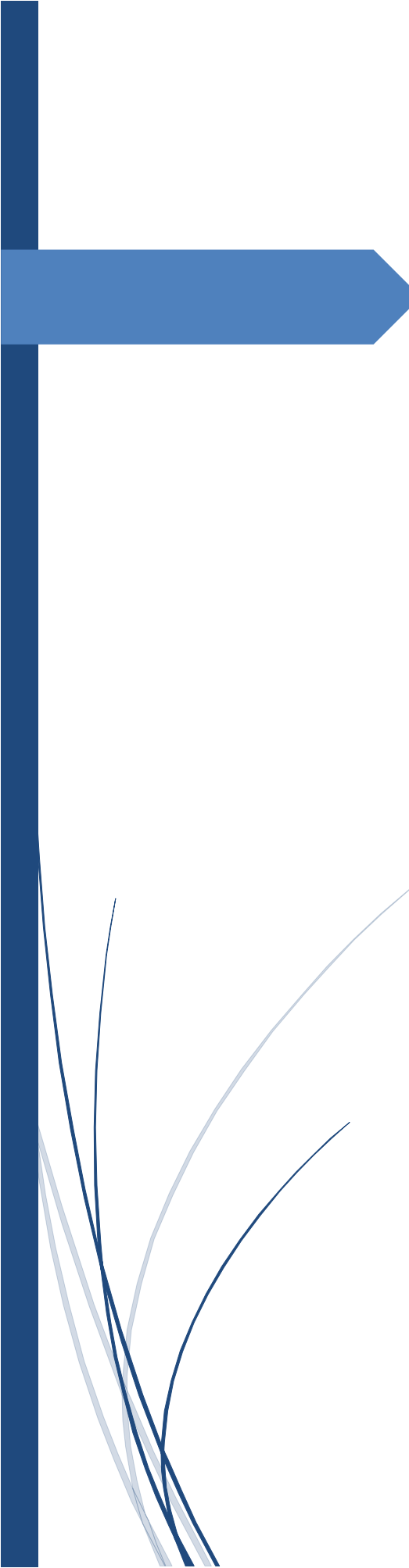


ANANYA VADLAMANI (8183302)

NAHAL NOUSHAD (8419395)

JONATHAN SAMUEL (8219047)

UOWD



ACKNOWLEDGMENTS

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this project.

First and foremost, I extend my deepest thanks to Dr. Mohamad Nassereddine and Mr. Mohamad Alrifai, whose guidance, support, and insightful feedback were invaluable throughout the project. Your expertise and encouragement greatly enhanced the quality of this work.

I am also grateful to the Engineering Department at University of Wollongong Dubai for providing the resources and facilities necessary for conducting this project. Special thanks to the technical staff and the IT department for their assistance with MATLAB and other necessary software tools.

Additionally, I would like to acknowledge my peers and classmates for their continuous support and for providing a collaborative environment. The discussions and brainstorming sessions were instrumental in refining the project.

Finally, I am profoundly thankful to my family and friends for their unwavering support and encouragement during this project.

Thank you all for your contributions and support.



Table of Contents

Chapter 1: Introduction and Background.....	3
1.1. Identification of the Problem:	3
1.2. List of Objectives:	3
1.3 List of Outcomes:	3
Chapter 2: Theoretical Design	4
2.1 Projectile Motion.....	4
2.2 Impacts of different conditions on the theoretical analysis	7
Chapter 3: MATLAB Coding	9
3.1 MATLAB Project.....	9
3.2 GUI Application of Code	15
Chapter 4: Conclusions	23
4.1. Purpose:.....	23
4.2. Approach:.....	23
4.3. Final Outcomes:	23
Chapter 5: References.....	24
Works Cited	24
Chapter 6: Appendix	25
6.1 List of Figures.....	25



Chapter 1: Introduction and Background

1.1. Identification of the Problem:

This project involves a thorough analysis of the trajectory of a projectile, initiated by the user from an origin point, aiming towards a basketball hoop situated at a specified distance and height relative to a building. The primary objective is to find the optimal launch velocity and angle required for the projectile to successfully clear the building and accurately land in the basketball hoop.

1.2. List of Objectives:

1. Develop a comprehensive analysis of the projectile's motion utilizing the equations of motion.
2. Implement MATLAB programming to calculate and plot the projectile's trajectory.
3. Create an application using MATLAB GUI that accepts input parameters D (distance to the building) and H (height of the building) and outputs the range of angles of projection and their corresponding velocities, time of flight and maximum height of the projectile.
4. Verify that the application can determine the projectile's position at any given moment in time.

1.3 List of Outcomes:

1. Conduct an analysis and develop an understanding of the basketball's projectile motion.
2. Develop a functional MATLAB program that accurately simulates the projectile's trajectory and determines the necessary launch parameters.
3. Create a user-friendly MATLAB application that provides real-time analysis and graphical representation of the projectile's path.
4. Ensure the capability to predict and verify the projectile's position and maximum height during its flight.

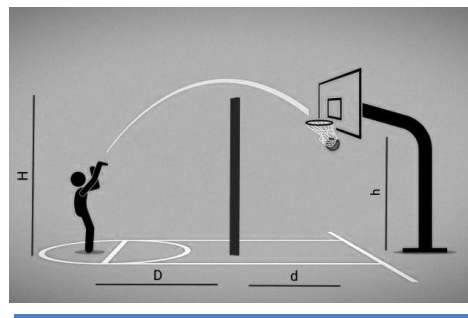


Figure 1: Rough Sketch of Plan



Chapter 2: Theoretical Design

2.1 Projectile Motion

Projectile motion can be referred to as the motion of an object thrown or projected into the air, subject to only the acceleration of gravity. The analysis of projectile motion involves separating values into their respective horizontal and vertical components and using the equations of motion to calculate each of the above-mentioned components.

Equations Involved in Calculations of the Dynamics:

$$v_x = v \times \cos(\theta)$$

v_x : horizontal component of velocity
 v : velocity

$$v_y = v \times \sin(\theta)$$

v_y : vertical component of velocity
 θ : angle of projection

The fundamental equations of motion are given by:

$$v = u + a \times t$$

u : initial velocity

$$s = u \times t + \frac{1}{2} \times a \times t^2$$

s : displacement of the projectile
 t : time taken during motion

$$v^2 - u^2 = 2 \times a \times s$$

a : acceleration of the projectile

In a projectile motion, the velocity of the horizontal component remains constant during the flight. The horizontal component of the acceleration therefore becomes zero. The following values are considered for the horizontal components of each factor of the projectile motion:

$$u_x = u \times \cos(\theta)$$

u_x : horizontal component of initial velocity

$$v_x = u \times \cos(\theta)$$

v_x : horizontal component of final velocity

$$a_x = 0$$

a_x : horizontal component of acceleration



In the case of vertical components, the velocity changes at every point of the motion. The acceleration acting in the y direction is acceleration due to gravity, which can approximately be considered as 9.81 m/s^2 acting downwards. At the maximum height of the projectile, the velocity becomes zero. The following values are considered for the vertical components of each factor of the projectile motion:

$$u_y = u \times \sin(\theta)$$

u_y : vertical component of initial velocity

$$v_y = 0$$

v_y : vertical component of final velocity at max height

$$a_y = -g$$

a_y : vertical component of acceleration

g : acceleration due to gravity

For this projectile, the following are known:

- The distance of the basketball hoop from the building (6 m)
- The height at which the basketball is placed (3 m)

The following values are taken by the user:

- The distance of the user from the building
- The height of the building

The position of the user is taken as the origin. Hence,

$$x_0 = 0$$

$$y_0 = 0$$

Let:

- D be the distance of the user from the building
- d be the distance of the basketball hoop from the building
- H be the height of the building
- h be the height at which the basketball is placed

$$R = D + d$$

$$R = \text{range}$$

d = distance from hoop to building

D = distance from user to building

R represents the range of the projectile, the distance between the user and the basketball hoop.

Deriving from the second equation of motion, the horizontal position can be given as:



$$\begin{aligned}
x &= x_0 + v_x \times t + \frac{1}{2} \times a_x \times t^2 \\
D + d &= 0 + v_x \times t + 0 \\
R &= v_x \times t \\
R &= v \times \cos(\theta) \times t
\end{aligned}$$

Hence t can be brought in terms of R and v as:

$$t = \frac{R}{v \times \cos(\theta)} \quad \text{— Eq. (1)}$$

Using the second equation of motion again, but for the vertical component of displacement, we can derive the equation,

$$\begin{aligned}
y &= y_0 + v_y \times t + \frac{1}{2} \times a_y \times t^2 \\
y &= 0 + v \times \sin(\theta) \times t - \frac{1}{2} \times g \times t^2 \\
y &= v \times \sin(\theta) \times t - \frac{1}{2} \times g \times t^2 \quad \text{— Eq. (2)}
\end{aligned}$$

The value of time, t, in Eq. (2) can be replaced by the equation derived from Eq. (1) as:

$$\begin{aligned}
y &= v \times \sin \theta \times \left(\frac{R}{v \times \cos(\theta)} \right) - \frac{1}{2} \times g \times \left(\frac{R}{v \times \cos(\theta)} \right)^2 \\
y &= R \times \left(\frac{\sin(\theta)}{\cos(\theta)} \right) - \frac{R^2 \times g}{2 \times v^2 \times \cos^2(\theta)} \\
y &= R \times \tan(\theta) - \frac{R^2 \times g}{2 \times v^2 \times \cos^2(\theta)} \\
\frac{R^2 \times g}{2 \times v^2 \times \cos^2(\theta)} &= R \tan(\theta) - y \\
v^2 &= \frac{R^2 \times g}{2 \times \cos^2(\theta) \times (R \times \tan(\theta) - y)} \\
v &= \sqrt{\frac{R^2 \times g}{2 \times \cos^2(\theta) \times (R \times \tan(\theta) - y)}} \quad \text{— Eq. (3)}
\end{aligned}$$

This equation can be used to calculate the initial velocity.

The value of velocity can be substituted back in Eq. (1) to find the total time of flight for the ball to start at the origin and end at the basketball hoop.

The value of velocity can also be used to calculate the time taken for the ball to reach the building, and the height of the ball at the position where the building is situated.

The time to building, t_b , can be found in the similar manner as the total time of flight, using the second equation of motion in the horizontal component as:



$$D = v_x \times t_b$$

$$D = v \times \cos(\theta) \times t_b$$

Hence t_b can be brought in terms of R and v as:

$$t_b = \frac{D}{v \times \cos(\theta)} \quad \text{— Eq.(4)}$$

The height of the ball at the position where the building is situated (y_b) can also be derived using the second equation of motion using vertical component as:

$$y = y_0 + v_y \times t + \frac{1}{2} \times a_y \times t^2$$

$$y_b = 0 + v_y \times t_b - \frac{1}{2} \times g \times t_b^2$$

$$y_b = v_y \times t_b - \frac{1}{2} \times g \times t_b^2 \quad \text{— Eq.(5)}$$

The maximum height of the projectile (h_{max}) can also be calculated using the third equation of motion as:

$$v_y^2 - u_y^2 = 2 \times a_y \times s_{y_{max}}$$

$$0 - (u \times \sin\theta)^2 = 2 \times -g \times h_{max}$$

$$h_{max} = \frac{u^2 \times \sin^2\theta}{2g} \quad \text{— Eq.(6)}$$

2.2 Impacts of different conditions on the theoretical analysis

The theoretical analysis of projectile motion is significantly affected by various conditions. Some of the factors are stated below:

2.2.1. Air Resistance:

The ideal analysis of the projectile motion ignores the effect of air resistance on the projectile. However, in real world scenarios air resistance decelerates the projectile motion and gives out a complex trajectory.

2.2.2. Gravity:

The primary condition of ideal projectile motion is that the object's vertical component of acceleration is exclusively due to gravity, which, on Earth, is approximately a constant value of 9.81 m/s².



2.2.3. Constant Horizontal velocity:

Another condition to be noted is that the horizontal component of velocity remains constant throughout the motion of the projectile. This occurs since the object experiences no acceleration in the horizontal direction.

2.2.4. Initial velocity:

The range and maximum height of the projectile is directly proportional to the initial velocity of the projectile.

2.2.5. Angle of projection:

The angle with which the projectile is being launched also played a great impact of the trajectory. Smaller angles result in lower trajectories and shorter flight durations, whereas larger angles (approaching 90°) produce higher trajectories with longer flight durations but reduced horizontal distances. 45° angle has been proved to provide the maximum range distance.

2.2.6. Initial height of projection:

Initial height of projectiles affects the projectile motion by increasing range distance and the time of flight as the projectile either must travel greater distance to land. Maximum height attained by the projectile would also vary.



Chapter 3: MATLAB Coding

3.1 MATLAB Project

3.1.1. Main Code:

```
%Getting the inputs from the user
mass=input('Enter mass of the ball (kg): ');
distance_from_building_user=input('Enter distance from the building (m): ');
height_of_building=input('Enter height of the building (m): ');
while 1
    if distance_from_building_user<=0 || height_of_building<=0
        disp('Values for height of the building and the distance from the building must be greater then 0')
        distance_from_building_user=input('Enter distance from the building (m): ');
        height_of_building=input('Enter height of the building (m): ');
    else
        break
    end
end

%Establishing the given information
ring_distance=6;
g=9.81;
ring_height=3;
range=distance_from_building_user+ring_distance;

%Finding the angle from the user end
min_angle_1=atan(height_of_building/distance_from_building_user);
disp('The minimum angle from user to building is: '+ rad2deg(min_angle_1));

%Finding the angle from the hoop end
min_angle_2=atan((height_of_building-ring_height)/ring_distance);
disp('The minimum angle from building to hoop is: '+rad2deg(min_angle_2));

%Checking which angle is greater and make that as the minimum angle
if min_angle_1>min_angle_2
    min_angle=min_angle_1;
else
    min_angle=min_angle_2;
end

%Calculating values
i=0;
for angle=min_angle:(pi/180):deg2rad(89)
    velocity=sqrt((range^2*g)/(2*((cos(angle))^2)*((range*tan(angle))-ring_height)));
    time_to_building=distance_from_building_user/(velocity*cos(angle));
```



```

height_proj_building=velocity*sin(angle)*time_to_building-0.5*g*time_to_building^2;
time_of_flight=range/(velocity*cos(angle));
height_reach_ring = (velocity*sin(angle)*time_of_flight) - (0.5*g*time_of_flight^2);
max_height_of_projectile = (velocity^2*(sin(angle))^2)/(2*g);
if (height_proj_building >= height_of_building)
    i=i+1;
    allangles.angle(i+1)=rad2deg(angle);
    allangles.velocity(i+1)=velocity;
    allangles.time(i+1)=time_of_flight;
    allangles.maxheight(i+1)=max_height_of_projectile;
else
    continue
end
end
end

%Plotting the graph
try
    single_angle = allangles.angle(2);
    single_velocity = allangles.velocity(2);
    single_time = allangles.time(2);
    single_height = allangles.maxheight(2);
    disp(['The angle taken is: ',num2str(single_angle)]);
    disp(['The initial velocity is: ',num2str(single_velocity)]);
    disp(['The time taken for the ball to reach the hoop is: ',num2str(single_time)]);
    disp(['In the complete projectile, the maximum height travelled by the ball is: ',
num2str(single_height)]);
    [x,y] = graphing(single_velocity,deg2rad(single_angle),single_time);
    plot(x,y,[distance_from_building_user distance_from_building_user],[0 height_of_building],
'LineWidth', 2);

%Calculations for certain time
user_spec_time=input('Do you want to enter specific time? (yes/no) ','s');
while 1
    if strcmp(user_spec_time, 'yes')
        specific_time = input("Enter the time between 0 to "+single_time+": ");
        while specific_time<0 || specific_time>single_time
            disp("Wrong Input. Error!");
            specific_time = input("Enter the time between 0 to "+single_time+": ");
        end
        [x_dist_st, y_dist_st, vel_at_time_st] = calc_at_specific_time(single_velocity,
deg2rad(single_angle), specific_time);
        disp(['At the specified time, the horizontal distance travelled is: ',num2str(x_dist_st)]);
        disp(['At the specified time, the vertical distance travelled is: ',num2str(y_dist_st)]);
        disp(['At the specified time, the velocity: ',num2str(vel_at_time_st)]);
        specific_time_graph = input("Graph? (yes/no) ","s");
        while 1
            if strcmp(specific_time_graph, 'yes')

```



```

        [x,y] = graphing(single_velocity,deg2rad(single_angle),specific_time);
        plot(x,y,[distance_from_building_user distance_from_building_user],[0
height_of_building'],'LineWidth',2);
        break
    elseif strcmp(specific_time_graph,'no')
        disp('THE END!!!');
        break
    else
        disp('Wrong Input. Error!')
        specific_time_graph = input("Graph? (yes/no) ", 's');
    end
end
break
elseif strcmp(user_spec_time,'no')
    disp('THE END!!!');
    break
else
    user_spec_time=input('Do you want to enter specific time?','s');
end
end
catch
    disp('Sorry, values are to extreme!')
end

```

3.1.2. Function 1: Graphing

```

function [x,y] = graphing(velocity, angle, timeofflight)
    t=0:0.001:timeofflight;
    x = velocity*cos(angle)*t;
    y = velocity*sin(angle)*t - 0.5*9.81*t.^2;
end

```

3.1.3. Function 2: Calculations for a User-Specified Time

```

function [x_dist, y_dist, vel_at_time] = calc_at_specific_time(velocity, angle, specific_time)
    x_dist = velocity*cos(angle)*specific_time;
    y_velocity_t = velocity*sin(angle)-9.81*specific_time;
    y_dist=velocity*sin(angle)*specific_time-0.5*9.81*specific_time^2;
    vel_at_time=sqrt(y_velocity_t^2+(velocity*cos(angle))^2);
end

```

3.1.4. Description for the Code

This MATLAB project is created in MATLAB script mode. The project, focused on calculating and displaying the optimum minimum angle to travel in a projectile motion without hitting the building situated in between the user and the hoop and successfully reaching the basketball hoop, begins with taking the input of



the mass of the ball, distance from the user to the building and the height of the building from the user.

Then, the values of some constants are saved into variables, such as the distance of the basketball hoop from the building, the height at which the basketball hoop is placed above the ground, and the value of constant acceleration due to gravity, which is roughly considered to be $9.81m/s^2$. Then, the value of distance between the user and the hoop is saved as the range, which is the sum of the distance between the user and the building and the distance between the building and the hoop.

Once the values are obtained and established, the next task done by the code of the project is to calculate the minimum angle required to pass above the building. The minimum angles considered are the angle from the user to the building and the angle from the building to the hoop. Since the values of the height and the base are known for both the conditions, the angles can be calculated using tan inverse function of MATLAB inbuilt library. Then, the greater of the two angles is considered as the minimum angle required to pass above the building.

The minimum angle to pass the building is found, however, it is not certain that the projectile does not hit the building during its time of flight. Hence, it is important to traverse through every angle between the found minimum angle and 90 degrees to find the range of angles which pass above the building successfully. The for loop is used to traverse through the angles within the range of the minimum angle and 90 degrees, with a step value of 1 degree.

For every angle, the velocity is calculated [using Eq.(3)] as derived above in Section 2.1. Using the velocity, the time taken to reach the building is calculated [using Eq.(4)], and using that, the height of the projectile at the point at which the building is placed [using Eq.(5)]. The value of the velocity is also used to calculate the total time of flight of the ball to reach the hoop [using Eq.(1)], the height of the ball at the end of the projectile [using Eq.(2)], which essentially has the same value as the height at which the basketball hoop is placed, and the maximum height travelled by the ball during its time of flight [using Eq.(6)].

An if condition is used to check if the height of the ball at the building is greater than height of the building. If it passes the condition, the angle is added to a structured array consisting of all the angles that pass the criteria, their corresponding velocity, time of flight and maximum height of the projectile.

An angle is chosen at random, which in this case, is the second angle of the structured array, and its corresponding velocity, time of flight and maximum height is chosen to plot the graph. To plot, a new function called graphing is created which



creates an array of the horizontal and vertical position for time intervals of 0.001s. This is then plotted using the plot() function, along with the straight line graph indicating the building.

The project then asks the user if they want to see the motion at a specific time. Upon entering yes, the user can enter a time between 0 to the time of flight of the projectile and then see the position of the ball at the specified time, its velocity and the graph. This is done using the user-defined function called calc_at_specific_time which calculates the horizontal distance travelled, vertical distance travelled and the velocity of the ball. Once again, the graphing and the plot function is used to plot the graph to the user-specified time.

```
Command Window
>> final_main_project_final
Enter mass of the ball (kg): 5
Enter distance from the building (m): 50
Enter height of the building (m): 100
The minimum angle from user to building is: 63.4349
The minimum angle from building to hoop is: 86.4604
The angle taken is: 87.4604
The initial velocity is: 78.8673
The time taken for the ball to reach the hoop is: 16.025
In the complete projectile, the maximum height travelled by the ball is: 316.4032
```

Figure 2: Output for the complete travel

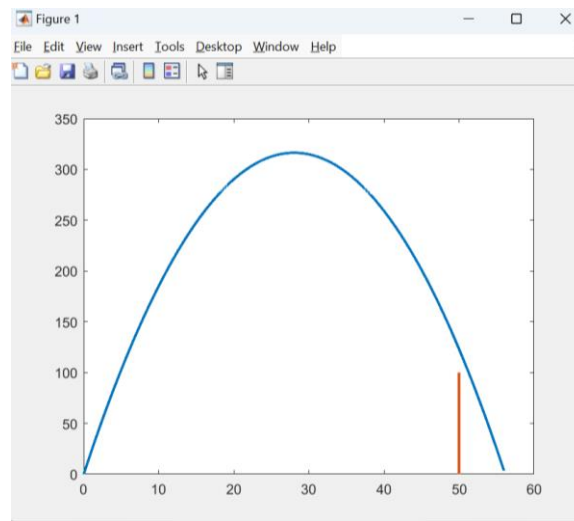


Figure 3: Trajectory for the whole flight



Command Window

```
>> final_main_project_final
Enter mass of the ball (kg): 5
Enter distance from the building (m): 50
Enter height of the building (m): 100
The minimum angle from user to building is: 63.4349
The minimum angle from building to hoop is: 86.4604
The angle taken is: 87.4604
The initial velocity is: 78.8673
The time taken for the ball to reach the hoop is: 16.025
In the complete projectile, the maximum height travelled by the ball is: 316.4032
Do you want to enter specific time? (yes/no) yes
Enter the time between 0 to 16.025: 14
At the specified time, the horizontal distance travelled is: 48.9236
At the specified time, the vertical distance travelled is: 141.6771
At the specified time, the velocity: 58.6544
Graph? (yes/no) yes
```

Figure 4: Output for the complete travel and for the user specified time

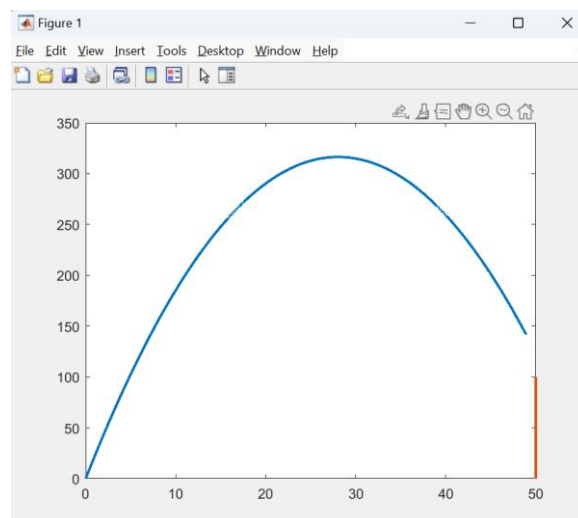


Figure 5: Trajectory up until the specified time



3.2 GUI Application of Code

3.2.1. GUI Code

```
classdef GUI_PROJECT < matlab.apps.AppBase
    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                matlab.ui.Figure
        TrajectoryofProjectileLabel matlab.ui.control.Label
        ProjectileatParticularTimeLabel matlab.ui.control.Label
        MovetheslidertoseethepositionandveloctiyoftheprojectilsLabel matlab.ui.control.Label
        UITable2                matlab.ui.control.Table
        Slider                  matlab.ui.control.Slider
        SliderLabel             matlab.ui.control.Label
        UITable                 matlab.ui.control.Table
        SelectAnglesListBox     matlab.ui.control.ListBox
        SelectAnglesListBoxLabel matlab.ui.control.Label
        EnterheightofthebuildingmEditField matlab.ui.control.NumericEditField
        EnterheightofthebuildingmEditFieldLabel matlab.ui.control.Label
        EnterdistancefrombuildingtotheusermEditField matlab.ui.control.NumericEditField
        EnterdistancefrombuildingtotheusermEditFieldLabel matlab.ui.control.Label
        CalculateButton         matlab.ui.control.Button
        Heightofbasketis3mLabel matlab.ui.control.Label
        Distancefrombaskettothebuildingis6mLabel matlab.ui.control.Label
        UIAxes2_2              matlab.ui.control.UIAxes
        UIAxes2                 matlab.ui.control.UIAxes
    end
    % Callbacks that handle component events
    methods (Access = private)
        % Button pushed function: CalculateButton
        function CalculateButtonPushed(app, event)
            distance_from_building_user= app.EnterdistancefrombuildingtotheusermEditField.Value;
            height_of_building=app.EnterheightofthebuildingmEditField.Value;
            ring_distance=6;
            g=9.81;
            ring_height=3;
            range=distance_from_building_user+ring_distance;
            min_angle_1=atan(height_of_building/distance_from_building_user);

            min_angle_2=atan((height_of_building-ring_height)/ring_distance);
            if min_angle_1>min_angle_2
                min_angle=min_angle_1;
            else
                min_angle=min_angle_2;
            end
            cla(app.UIAxes2);
            i=0;
```




```

for angle=deg2rad(ceil((rad2deg(min_angle))):(pi/180):deg2rad(89)
    velocity=sqrt((range^2*g)/(2*((cos(angle))^2)*((range*tan(angle))-ring_height)));
    time_to_building=distance_from_building_user/(velocity*cos(angle));
    height_proj_building=velocity*sin(angle)*time_to_building-0.5*g*time_to_building^2;
    time_of_flight=range/(velocity*cos(angle));
    height_reach_ring = (velocity*sin(angle)*time_of_flight) - (0.5*g*time_of_flight^2);
    max_height_of_projectile = (velocity^2*(sin(angle))^2)/(2*g);
    if (height_proj_building >= height_of_building)
        i=i+1;
        allangles.angle(i+1)=rad2deg(angle);
        allangles.velocity(i+1)=velocity;
        allangles.time(i+1)=time_of_flight;
        allangles.maxheight(i+1)=max_height_of_projectile;
        app.SelectAnglesListBox.Items(i+1)={num2str(rad2deg(angle))};
    else
        continue
    end
end
if exist("allangles",'var')==0
    errordlg('Values too Extreme!')
end
end
% Clicked callback: SelectAnglesListBox
function SelectAnglesListBoxClicked(app, event)
    distance_from_building_user= app.EnterdistancefrombuildingtotheusermEditField.Value;
    height_of_building=app.EnterheightofthebuildingmEditField.Value;
    ring_distance=6;
    g=9.81;
    ring_height=3;
    range=distance_from_building_user+ring_distance;

    min_angle_1=atan(height_of_building/distance_from_building_user);
    min_angle_2=atan((height_of_building-ring_height)/ring_distance);
    if min_angle_1>min_angle_2
        min_angle=min_angle_1;
    else
        min_angle=min_angle_2;
    end

    allangles = app.SelectAnglesListBox.Items;
    main_angle = deg2rad(str2double(allangles(event.InteractionInformation.Item)));
    single_velocity=sqrt((range^2*g)/(2*((cos(main_angle))^2)*((range*tan(main_angle))-
ring_height)));
    single_time=range/(single_velocity*cos(main_angle));
    single_height=(single_velocity^2*(sin(main_angle))^2)/(2*g);

    values(1,1)=rad2deg(main_angle);

```



```

values(1,2)=single_velocity;
values(1,3)=single_time;
values(1,4)=single_height;

app.UITable.Data=values;

[x,y] = graphing(single_velocity,main_angle,single_time);
hold(app.UIAxes2, 'on')
for nf = 0:(length(x))/30:length(x)

    plot(app.UIAxes2, x(1:nf),y(1:nf),[distance_from_building_user
distance_from_building_user],[0 height_of_building], "LineWidth", 2);
    pause(0.001);
end
end
% Value changing function: Slider
function SliderValueChanging(app, event)
    changingValue = event.Value;
    time_for_slider=app.UITable.Data(1,3);
    app.Slider.Limits=[0 time_for_slider];
end
% Value changed function: Slider
function SliderValueChanged(app, event)
    distance_from_building_user= app.EnterdistancefrombuildingtotheusermEditField.Value;
    height_of_building=app.EnterheightofthebuildingmEditField.Value;
    ring_distance=6;
    g=9.81;
    ring_height=3;
    range=distance_from_building_user+ring_distance;
    specific_time = app.Slider.Value;
    range_time_specific=app.UITable.Data(1,2);
    time_specific_=app.UITable.Data(1,1);
    [x_dist_st, y_dist_st, vel_at_time_st] = calc_at_specific_time(range_time_specific,
deg2rad(time_specific_), specific_time);
    values_for_2nd_table(1,1)=specific_time;
    values_for_2nd_table(1,2)=x_dist_st;
    values_for_2nd_table(1,3)=y_dist_st;
    values_for_2nd_table(1,4)=vel_at_time_st;
    app.UITable2.Data=values_for_2nd_table;

    [x_for_specific_time the_rock] =
graphing(range_time_specific,deg2rad(time_specific_),specific_time);
    plot(app.UIAxes2_2,x_for_specific_time,the_rock,[distance_from_building_user
distance_from_building_user],[0 height_of_building], 'LineWidth',2);

end
end

```



```

% Component initialization
methods (Access = private)
% Create UIFigure and components
function createComponents(app)
    % Create UIFigure and hide until all components are created
    app.UIFigure = uifigure('Visible', 'off');
    app.UIFigure.Position = [100.333333333333 100.333333333333 1090 741];
    app.UIFigure.Name = 'MATLAB App';
    % Create UIAxes2
    app.UIAxes2 = uiaxes(app.UIFigure);
    title(app.UIAxes2, 'Flight of Projectile')
    xlabel(app.UIAxes2, 'X (m)')
    ylabel(app.UIAxes2, 'Y (m)')
    zlabel(app.UIAxes2, 'Z')
    app.UIAxes2.Position = [591 372 440 290];
    % Create UIAxes2_2
    app.UIAxes2_2 = uiaxes(app.UIFigure);
    title(app.UIAxes2_2, 'Trajectory')
    xlabel(app.UIAxes2_2, 'X (m)')
    ylabel(app.UIAxes2_2, 'Y (m)')
    zlabel(app.UIAxes2_2, 'Z')
    app.UIAxes2_2.Position = [591 68 442 290];
    % Create Distancefrombaskettothebuildingis6mLabel
    app.Distancefrombaskettothebuildingis6mLabel = uilabel(app.UIFigure);
    app.Distancefrombaskettothebuildingis6mLabel.Position = [49 535 236 22];
    app.Distancefrombaskettothebuildingis6mLabel.Text = 'Distance from basket to the building is:
6 m';
    % Create Heightofbasketis3mLabel
    app.Heightofbasketis3mLabel = uilabel(app.UIFigure);
    app.Heightofbasketis3mLabel.Position = [49 510 130 22];
    app.Heightofbasketis3mLabel.Text = 'Height of basket is: 3 m';
    % Create CalculateButton
    app.CalculateButton = uibutton(app.UIFigure, 'push');
    app.CalculateButton.ButtonPushedFcn = createCallbackFcn(app, @CalculateButtonPushed,
true);
    app.CalculateButton.Position = [292 500 100 22];
    app.CalculateButton.Text = 'Calculate';
    % Create EnterdistancefrombuildingtotheusermEditFieldLabel
    app.EnterdistancefrombuildingtotheusermEditFieldLabel = uilabel(app.UIFigure);
    app.EnterdistancefrombuildingtotheusermEditFieldLabel.HorizontalAlignment = 'right';
    app.EnterdistancefrombuildingtotheusermEditFieldLabel.Position = [42 590 237 22];
    app.EnterdistancefrombuildingtotheusermEditFieldLabel.Text = 'Enter distance from building
to the user (m)';
    % Create EnterdistancefrombuildingtotheusermEditField
    app.EnterdistancefrombuildingtotheusermEditField = uieditfield(app.UIFigure, 'numeric');
    app.EnterdistancefrombuildingtotheusermEditField.HorizontalAlignment = 'left';
    app.EnterdistancefrombuildingtotheusermEditField.Position = [295 592 100 20];

```



```

% Create EnterheightofthebuildingmEditFieldLabel
app.EnterheightofthebuildingmEditFieldLabel = uilabel(app.UIFigure);
app.EnterheightofthebuildingmEditFieldLabel.HorizontalAlignment = 'right';
app.EnterheightofthebuildingmEditFieldLabel.Position = [44 560 172 22];
app.EnterheightofthebuildingmEditFieldLabel.Text = 'Enter height of the building (m)';
% Create EnterheightofthebuildingmEditField
app.EnterheightofthebuildingmEditField = uieditfield(app.UIFigure, 'numeric');
app.EnterheightofthebuildingmEditField.HorizontalAlignment = 'left';
app.EnterheightofthebuildingmEditField.Position = [293 562 100 20];
% Create SelectAnglesListBoxLabel
app.SelectAnglesListBoxLabel = uilabel(app.UIFigure);
app.SelectAnglesListBoxLabel.HorizontalAlignment = 'right';
app.SelectAnglesListBoxLabel.Position = [64 480 75 22];
app.SelectAnglesListBoxLabel.Text = 'SelectAngles';
% Create SelectAnglesListBox
app.SelectAnglesListBox = uilistbox(app.UIFigure);
app.SelectAnglesListBox.Items = {'0'};
app.SelectAnglesListBox.ClickedFcn = createCallbackFcn(app, @SelectAnglesListBoxClicked,
true);
app.SelectAnglesListBox.Position = [54 400 100 74];
app.SelectAnglesListBox.Value = {};
% Create UITable
app.UITable = uitable(app.UIFigure);
app.UITable.ColumnName = {'ANGLE (°)'; 'VELOCITY (m/s)'; 'TIME (s)'; 'MAX. HEIGHT
(m)'};
app.UITable.RowName = {};
app.UITable.Position = [177 402 385 72];
% Create SliderLabel
app.SliderLabel = uilabel(app.UIFigure);
app.SliderLabel.HorizontalAlignment = 'right';
app.SliderLabel.Position = [30 212 36 22];
app.SliderLabel.Text = 'Slider';
% Create Slider
app.Slider = uislider(app.UIFigure);
app.Slider.ValueChangedFcn = createCallbackFcn(app, @SliderValueChanged, true);
app.Slider.ValueChangingFcn = createCallbackFcn(app, @SliderValueChanging, true);
app.Slider.Position = [87 221 427 3];
% Create UITable2
app.UITable2 = uitable(app.UIFigure);
app.UITable2.ColumnName = {'TIME (s)'; 'RANGE (m)'; 'HEIGHT (m)'; 'VELOCITY (m/s)'};
app.UITable2.RowName = {};
app.UITable2.Position = [36 83 488 61];
% Create MovetheslidertoseethepositionandveloctiyoftheprojectilsLabel
app.MovetheslidertoseethepositionandveloctiyoftheprojectilsLabel = uilabel(app.UIFigure);
app.MovetheslidertoseethepositionandveloctiyoftheprojectilsLabel.Position = [35 248 452 22];
app.MovetheslidertoseethepositionandveloctiyoftheprojectilsLabel.Text = 'Move the slider to
see the position and veloctiy of the projectile at a particular time.';

```



```

    % Create ProjectileatParticularTimeLabel
    app.ProjectileatParticularTimeLabel = uilabel(app.UIFigure);
    app.ProjectileatParticularTimeLabel.FontName = 'Times New Roman';
    app.ProjectileatParticularTimeLabel.FontSize = 24;
    app.ProjectileatParticularTimeLabel.FontWeight = 'bold';
    app.ProjectileatParticularTimeLabel.Position = [35 268 325 53];
    app.ProjectileatParticularTimeLabel.Text = 'Projectile at Particular Time';
    % Create TrajectoryofProjectileLabel
    app.TrajectoryofProjectileLabel = uilabel(app.UIFigure);
    app.TrajectoryofProjectileLabel.FontName = 'Times New Roman';
    app.TrajectoryofProjectileLabel.FontSize = 36;
    app.TrajectoryofProjectileLabel.FontWeight = 'bold';
    app.TrajectoryofProjectileLabel.Position = [49 635 369 53];
    app.TrajectoryofProjectileLabel.Text = 'Trajectory of Projectile';
    % Show the figure after all components are created
    app.UIFigure.Visible = 'on';
end
end
% App creation and deletion
methods (Access = public)
    % Construct app
    function app = GUI_PROJECT
        % Create UIFigure and components
        createComponents(app)
        % Register the app with App Designer
        registerApp(app, app.UIFigure)
        if nargin == 0
            clear app
        end
    end
end
% Code that executes before app deletion
function delete(app)
    % Delete UIFigure when app is deleted
    delete(app.UIFigure)
end
end
end
end

```



3.2.2. Explanation for the GUI

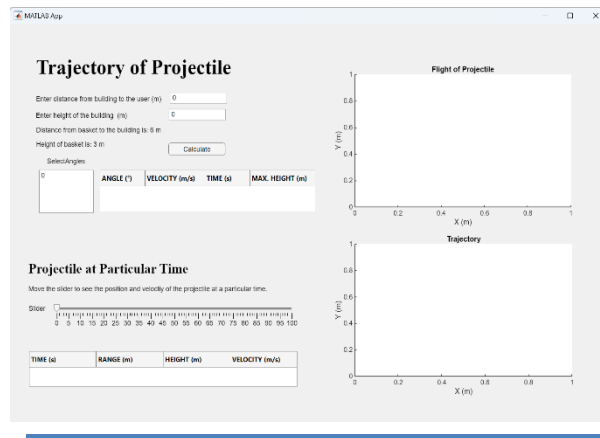


Figure 6: GUI Interface

The GUI is built on the basis of the already existing main project. On the design view, edit fields are placed to obtain the input of distance of building from the user and height of the building. These values are extracted from the design view onto the code view and saved as variables along with the values of the constants used in the project.

Upon clicking the calculate button, using the same logic as the main code, the angles that satisfy the criteria of reaching the hoop without hitting the building are saved into the structured array along with their corresponding velocities, time of flight and maximum height of the projectile. The list of range of angles that satisfy the criteria are displayed onto the list box which allows the user to select and view its corresponding velocity, time of flight and maximum height and displays an animated graph using these values.

The second part of the app allows users to move the slider between 0 and time of flight of the projectile and displays the horizontal position, the vertical position and the velocity of the ball at a specified time and displays the graph of the projectile motion until the specified time.



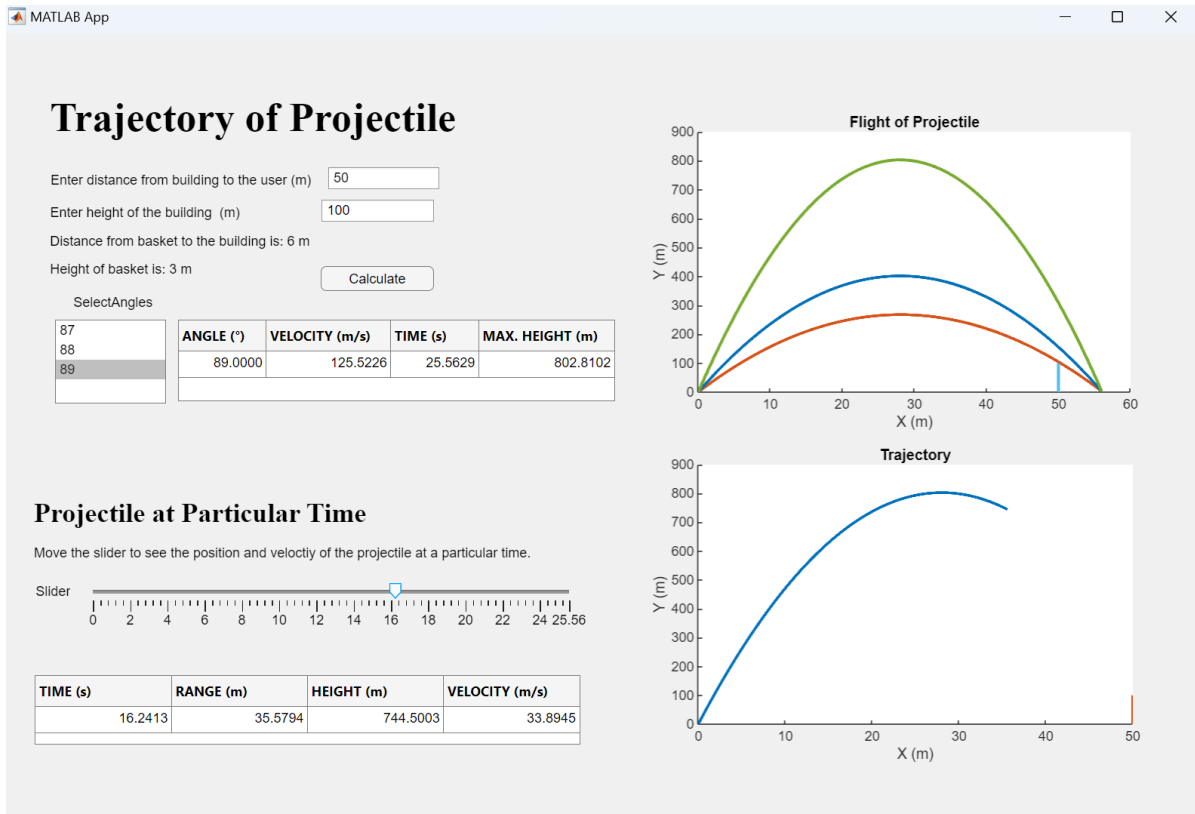


Figure 7: GUI Interface with Values and Graph



Chapter 4: Conclusions

4.1. Purpose:

The project focuses on mapping the trajectory of a basketball (projectile) to a basketball ring located 6 meters beyond a building and 3 meters above the ground. The projectile's launch point is situated at a distance D from the building, which has a height H . The project involves applying the equations of motion to determine the optimal angle of projection (θ) and initial velocity (V) required for the basketball to successfully reach the ring.

4.2. Approach:

The project uses principles of projectile motion dynamics to model to simulate the trajectory of a basketball. The fundamental equations of motion are applied through MATLAB programming to analyze and visualize the path of the projectile. The project includes functions to calculate the trajectory, determine the velocity, time of flight, and maximum height, and verify the landing position relative to the basketball ring. A MATLAB GUI application enhances user interaction by allowing the input of distance D and height H to compute and display optimal launch parameters such as velocity V and angle θ . The application also provides graphing capabilities, available both within the GUI and the main MATLAB project for visual representations of the projectile's flight path.

4.3. Final Outcomes:

Upon completion, the project provides a MATLAB code capable of accurately simulating and visualizing the dynamics of projectile motion. The code enables the user to analyze the object's movement by listing out the object's initial velocity, angle of projection, maximum height attained and the time of flight of the projectile. The GUI offers users an array of optimal angles that can be utilized for plotting graphs. It is designed to be highly user-friendly, featuring an animation that displays the basketball's motion from launch to the hoop. Additionally, the GUI includes a slider, allowing users to select any specific timeframe to determine the basketball's location at that moment. The experience has sharpened problem-solving abilities through refining code for accurate trajectory modeling. This project underscored the practical applications of projectile motion analysis across disciplines, highlighting its relevance in sports, engineering, and physics



Chapter 5: References

Works Cited

1. Lumen Learning. "Projectile Motion | Physics." *Courses.lumenlearning.com*, Lumen Learning, 2022, courses.lumenlearning.com/suny-physics/chapter/3-4-projectile-motion/.
2. Openstax. "5.3 Projectile Motion - Physics | OpenStax." *Openstax.org*, 2023, openstax.org/books/physics/pages/5-3-projectile-motion#:~:text=Projectile%20motion%20is%20the%20motion.
3. Study.com. "Describe Conditions for Ideal Projectile Motion Using a Free-Falling Body." Study.com.
4. MathWorks. "ReferenceEntities." Mathworks.com, 2019, www.mathworks.com/help/matlab/.
5. "MATLAB GUI." Mathworks.com, 2019, www.mathworks.com/discovery/matlab-gui.html.
6. "Subcategories." Mathworks.com, 2019, www.mathworks.com/help/simulink/.



Chapter 6: Appendix

6.1 List of Figures

Chapter 1

1.1.3

Figure 1 Plan of the project

Chapter 3

3.1.4.

Figure 2 Output for the whole trajectory

Figure 3 Graph for the whole trajectory

Figure 4 Output for the user specified time

Figure 5 Graph for the up until the user specified time

3.2.2.

Figure 6 User Interface

Figure 7 User Interface with Values

